

TECHNICAL DOCUMENTATION

Nifty Oracle

A free, honest intraday forecasting system for BankNifty & Nifty50 — architecture, models, indicators, APIs, and the discipline that keeps it real.

Version 3.1 · Engine: forecast_engine v3.1 + ORB daily engine
Data: Angel One SmartAPI (real-time) + Yahoo Finance (fallback)
Compiled: 26 July 2026 · Location: ~/Desktop/QUANTSSSS

⚠ **The one truth that matters:** real out-of-sample accuracy is **~50–59%**, never 90%+. This document exists partly to prove that number honestly. Anyone — human or AI — promising guaranteed profit or 90% accuracy is lying. This system survives on selectivity and risk management, not prophecy.

§ Contents

1	Executive summary & the honest truth	01
2	System architecture	02
3	Data layer & caching	03
4	Feature engineering (the 13 indicators)	04
5	The models — LightGBM & GradientBoosting	05
6	Confidence tiers (HIGH / MEDIUM / LOW)	06
7	The STRONG signal — consensus + time filter	07
8	The ORB daily engine	08
9	The AI layer (Gemini analyst & co-pilot)	09
10	The dashboard — routes, APIs, UI	10
11	Backtesting & honest results	11
12	What was tested and removed	12
13	APIs reference (Angel, Upstox, Gemini, Yahoo)	13
14	Daily operating guide	14
15	Risk management & limitations	15
16	Statistical methods — what we kept & removed	16
17	File-by-file reference & glossary	17

1 Executive summary & the honest truth

Nifty Oracle is a self-hosted, zero-cost intraday forecasting system for two Indian index instruments — **BankNifty** and **Nifty50**. It answers one question every minute: *which way is the index likely to move, how sure are we, and is now a good time to act?* It never places live orders; it forecasts, paper-trades, and enforces risk discipline.

Update — 28 July 2026 (supersedes stale details below).

- **Single engine.** The project was consolidated to **one** forecast engine, `forecast_engine.py`. The legacy "Beast" (VIX/cross-index) and the volatility engine were moved to `experiments/`; the Upstox integration was deleted (Angel One is the only live feed). See `STRUCTURE.md`.
- **Head-to-head verdict.** New vs legacy, logged live on Angel data, had **no persistent winner** — legacy led in the morning and reversed by the close (BankNifty 60m went $-23 \rightarrow +22$ in one day). Single-window numbers flip; that is noise, not edge.
- **Accuracy $\neq$ profit.** A cost-aware **A+ selective backtest** (STRONG + prime window + full trend alignment, adaptive ATR stops, slippage, one entry per setup) on 28 Jul **lost money on both indices** — BankNifty $-\text{₹}4,629$ (profit factor 0.20), Nifty50 $-\text{₹}3,300$ (0.52). A typical 30-min move ($\sim 36 / \sim 14$ pts) barely exceeds round-trip cost+slippage ($\sim 27 / \sim 10$ pts), and per-minute hit-rates are inflated $\sim 8\text{--}11\times$ by autocorrelation. Scored as distinct trades, the edge disappears.
- **Security.** Secrets live only in `.env`. The Angel TOTP secret is a live 2FA bypass and a `.gitignore` does not protect a zipped folder — if it was ever shared, rotate the API key and reset TOTP.

Update — 30 July 2026 (data breakthrough + the exhaustive investigation).

- **Real data unlocked, free.** The index spot has volume=0 (its "VWAP" is fake). We now pull index **futures with real volume** from Angel (`fetch_futures.py` : ~2.5yr daily, ~3mo 5-min) and **2 years of per-strike option EOD with Open Interest** from the NSE F&O bhavcopy (`fetch_bhavcopy.py` ; free, not geo-blocked). No IV (compute from premium); intraday per-strike OI remains paid-only.
- **100+ strategies tested** (price, OI/PCR, cross-index, money-flow, regime, ensemble, volatility, VRP) — all per-day / out-of-sample with half-split stability. **Nothing beats ~53-54% direction robustly.** Gap-fade looked 62% but per-day = 45% (autocorrelation). PCR-extremes hit 58% and was briefly cost-positive, then **decayed and died** (last 6 mo: 36%, PF 0.74). Volatility prediction is the one thing that holds (~54-65% per horizon) — but it forecasts *magnitude, not direction*.
- **VRP is real but not retail-tradeable.** Short ATM straddles win 62-72% (premium > realized move) — a classic **Taleb distribution**: retail option costs eat the edge and a single bad week (– ₹50-70k) can erase months. The defined-risk (iron-fly) version caps the tail but costs then push net to ~breakeven/loss. VRP is the market-maker's edge (their cost ≈ 0).
- **Conclusion (theory + our data agree).** A high win-rate is a warning sign of hidden tail risk, not skill; short-horizon prediction is dominated by microstructure noise; the only real micro-edge is HFT order-book / latency arbitrage (needs colocation). Chase **expectancy**, not accuracy. Treat this system as honest information / learning, not an income source.

DATA COST

₹0

HISTORY

~17 mo

ML MODELS

8

INDICATORS

13

STRONG ACC.

54–59%

Two engines, one philosophy

- **Forecast Window** (`forecast_engine.py`) — an every-minute, multi-horizon direction forecaster (5m / 15m / 30m / 60m) built on 8 LightGBM models. Surfaces a ★ **STRONG** signal only when several horizons agree with high confidence during the model's reliable hours.
- **ORB daily engine** (now archived in `experiments/nifty_oracle.py`) — a daily-bias model gating an opening-range breakout. **A full-history backtest found no robust edge** (see §11), so it was moved out of the live product. `nifty_live.py` stays only as a shared indicator/data utility.

The honest truth (repeated on purpose). Per-bar direction prediction on liquid index data is close to a coin toss (~50–53%). The edge is small and comes from **selectivity** — acting only on the strongest, best-timed setups (~54–59%) — combined with a **1.5 : 1 reward-to-risk** ratio. There is no configuration, data source, or model that reaches 90%. Chasing that number is how retail traders lose money.

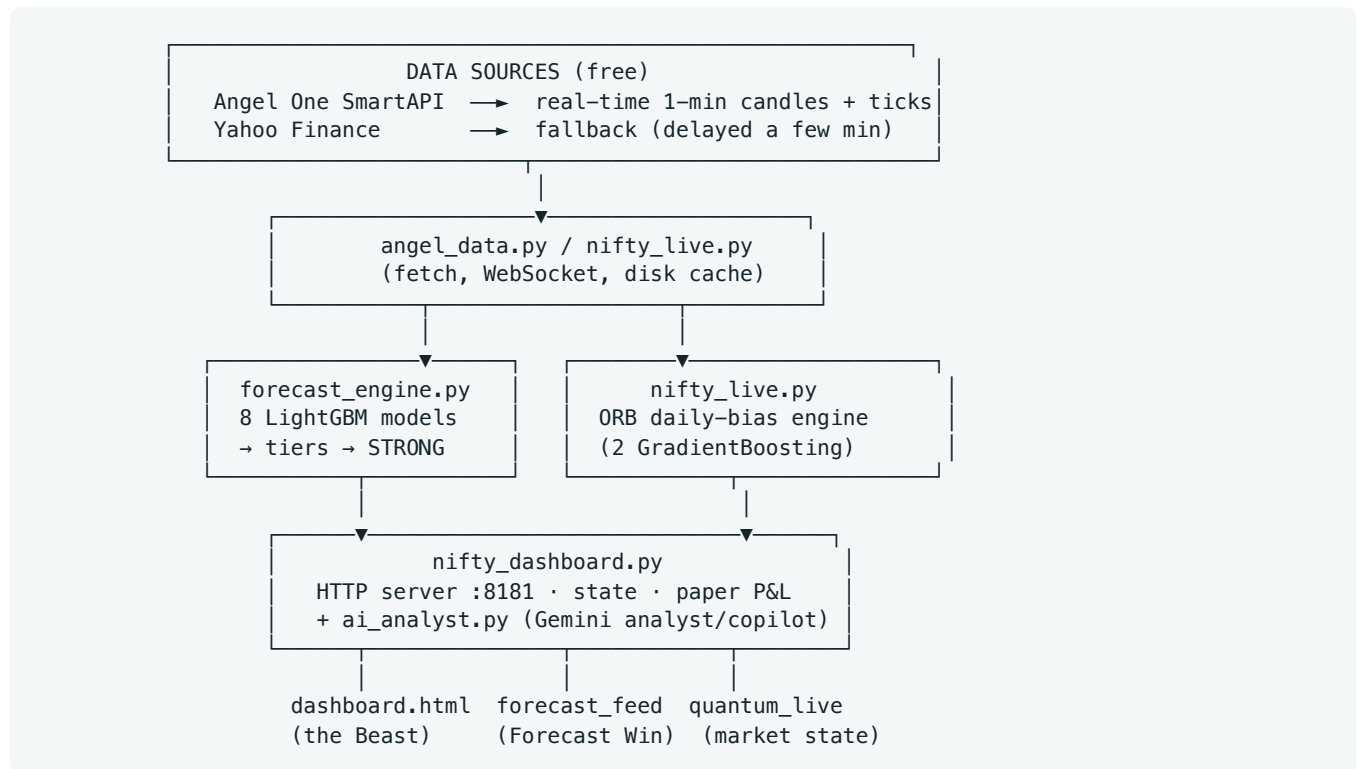
Design principles (the iron rules)

1. **No feature ships without a walk-forward backtest proving it helps.** Plausibility is not evidence.

2. **Train/serve consistency.** Every feature is computed identically in training and live — no lookahead, no data that only exists after the fact.
3. **Honesty over comfort.** The UI shows the real hit-rate, labelled "not a guarantee." Losing signals are not hidden.
4. **Selectivity beats prediction.** Sitting out (WAIT) is a first-class decision, not a failure.

2 System architecture

The system is a single-process Python web server that streams market data, computes features, runs the models, and serves three HTML dashboards. There is no database — state lives in memory and a few JSON/CSV files on disk.



File inventory (the live system)

File	Lines	Role
nifty_dashboard.py	902	HTTP server, state, paper trading, API routes, orchestration
forecast_engine.py	382	Multi-horizon LightGBM forecaster, tiers, STRONG signal
nifty_live.py	443	ORB daily-bias engine (live / replay / backtest), shared helpers
nifty_oracle.py	298	Morning-briefing CLI (daily bias, pivots, ORB plan)
angel_data.py	167	Angel One SmartAPI: auto-login (TOTP), candles, WebSocket ticks
upstox_data.py	179	Upstox OAuth + candle feed (optional backup)
upstox_autologin.py	76	Playwright auto-login stub (WIP, for friend's account)
ai_analyst.py	139	Gemini technical analyst + co-pilot verdict
backtest.py	74	Honest walk-forward accuracy (per horizon + per tier)
train_models.py	40	Pre-train / retrain the forecast models
dashboard.html	514	"The Beast" — live chart, paper trades, P&L
forecast_feed.html	257	Forecast Window — horizons, tiers, STRONG/WAIT banner
quantum_live.html	220	Market-state view — 7 indicator chips

Deprecated/experimental files (broken engines, one-off scripts, pre-project cruft) were moved to `_archive/` during the July 2026 cleanup — not deleted, so anything can be recovered.

3 Data layer & caching

All data is free. The feed priority is **Angel One** → **Yahoo**. Angel is preferred because it is real-time and — crucially — logs in via **TOTP** (a local authenticator secret), so it works from anywhere without an SMS OTP.

Sources

Source	Use	Notes
Angel One SmartAPI	Primary real-time feed	1-min candles via <code>getCandleData</code> ; live ticks via <code>SmartWebSocketV2</code> (mode 1 LTP). Auto-login: API key + PIN + TOTP-generated OTP.
Yahoo Finance	Always-on fallback	<code>yfinance</code> ; tickers <code>^NSEBANK</code> , <code>^NSEI</code> . Can lag a few minutes and rate-limits under heavy polling.
Upstox	Optional backup	OAuth token expires daily (~3:30 AM). Needs SMS OTP → not used by default.

On-disk cache (`data_cache/`)

File	Contents
<code>BANKNIFTY_5m_long.csv</code> <code>NIFTY50_5m_long.csv</code>	~26,000 rows of 5-min OHLC, Feb 2025 → Jul 2026 (~17 months, ~351 sessions). The training set.
<code>INDIAVIX_5m_long.csv</code>	India VIX 5-min history (was used by the removed "extras" experiment).
<code>*_daily.csv</code>	Daily bars for the ORB bias model (2 years).
<code>*_5m.csv</code>	Per-day intraday cache written during sessions.
<code>*_models_v2.pkl</code>	Pickled trained models + tiers + STRONG stats, keyed by data-date and version (v3.1). First train ≈ 2–3 min, then instant load.

`forecast_history_v2.json` stores every forecast with its ✓/✗ outcome per horizon, so the UI can show today's live hit-rate.

4 Feature engineering — the 13 indicators

The forecast models consume **13 features** (`BASE_COLS`), all computed from 5-minute OHLC bars and all **causal** (only past/current data). This self-consistency is deliberate: every feature means the same thing in training and in live inference.

#	Feature	What it captures	Formula (essence)
1	<code>r1</code>	1-bar return / momentum	<code>close.pct_change()</code>
2	<code>rsi</code>	Relative Strength Index (14)	$100 - 100/(1+RS)$
3	<code>ema20</code>	Distance from EMA-20 (trend)	<code>close/EMA20 - 1</code>
4	<code>macd_hist</code>	MACD histogram (momentum shift)	$(MACD - signal)/close$
5	<code>bb_width</code>	Bollinger band width (volatility spread)	$2 \cdot \sigma_{20} / MA_{20}$
6	<code>bb_pos</code>	Bollinger %b (position in band)	$(close - MA_{20}) / (2 \cdot \sigma_{20})$
7	<code>atr_pct</code>	ATR as % of price (volatility)	<code>ATR14 / close</code>
8	<code>parkinson_vol</code>	High-low range volatility	$\sqrt{(\text{mean}(\ln(h/l)^2)/(4\ln 2))}$
9	<code>vwap</code>	Distance from session VWAP	<code>close/VWAP - 1</code>
10	<code>tod</code>	Time of day (session phase)	$(min - 555)/375$
11	<code>dow</code>	Day of week	<code>weekday / 4</code>
12	<code>regime_vol</code>	Rolling volatility of returns	<code>std(r1, 20)</code>
13	<code>hurst</code>	Hurst exponent (trend vs range)	$\log(R/S)/\log(20)$

Why tree models don't need feature de-correlation. Several features overlap (EMA, MACD, RSI all encode momentum). Linear models would suffer from this multicollinearity; gradient-boosted trees simply pick the best split at each node and are unaffected. This is why no PCA / copula / whitening step is needed or used.

5 The models — LightGBM & GradientBoosting

Ten trained models total. Both algorithms are **gradient boosting** — an ensemble of small decision trees where each new tree corrects the previous trees' errors. The difference is implementation, not idea.

	LightGBM	GradientBoosting (sklearn)
Used for	Forecast Window (8 models)	ORB daily bias (2 models)
Data size	~26,000 rows (big)	~500 days (small)
Why	Fast, handles large data, leaf-wise growth	Stable on small data, proven, less overfit-prone

5.1 Forecast models (8 × LightGBM)

For each index (BankNifty, Nifty50) and each horizon (5m / 15m / 30m / 60m) a separate binary classifier predicts $P(\text{price higher after } k \text{ bars})$. Label: $y = 1 \text{ if } \text{close}(t+k) > \text{close}(t) \text{ else } 0$

```
LGBMClassifier(  
    objective="binary", num_leaves=24, max_depth=4,  
    learning_rate=0.02, n_estimators=500,  
    subsample=0.75, colsample_bytree=0.7,  
    reg_alpha=0.5, reg_lambda=2.0,          # strong regularisation  
    min_child_samples=80)                  # → resists overfitting noise
```

Training uses a held-out validation slice (last ~10% of sessions) with early stopping, then retrains on all data for production. Horizons in 5-min bars: 5m→1, 15m→3, 30m→6, 60m→12.

5.2 Daily-bias models (2 × GradientBoosting)

Predicts whether the day closes above its open. Features are daily: returns (1/3/5-day), RSI, EMA-20/50 distance, ATR%, range position, gap, day-of-week, plus (BankNifty only) MACD, Bollinger %b, Stochastic. Nifty50 stays lean — indicator features overfit it.

```
GradientBoostingClassifier(n_estimators=150, max_depth=3,  
                           learning_rate=0.05, subsample=0.8)  
FEATURE_SETS = {"BANKNIFTY": BASE+INDICATORS (13),  
                "NIFTY50":  BASE (10)}
```

Candlestick patterns (doji, engulfing, hammer) are computed for the UI but are **not** model inputs — they did not improve accuracy.

6 Confidence tiers — HIGH / MEDIUM / LOW

A raw probability like 0.53 is nearly meaningless on its own. Instead each forecast is bucketed into a tier whose **real held-out hit-rate is measured and displayed**. Thresholds are learned per horizon from the validation slice — not hard-coded — so the tiers actually fire.

Tier	Definition	Typical held-out accuracy
HIGH	Top ~28% most-confident calls (conf $\geq$ 72nd percentile)	~53–57%
MEDIUM	Next ~35%	~49–53%
LOW	Remaining ~37%	~48–51% (coin toss)

Calibration (`_calibrate_tiers`) computes the confidence distribution on the validation set, sets the HIGH/MEDIUM cutoffs by percentile, and records each bucket's accuracy. Only the **HIGH** tier carries a usable edge; MEDIUM and LOW are shown for transparency but should not be traded alone.

Key insight. Accuracy does not come from predicting more — it comes from predicting *less*, but only when confident. The tier system is how the engine says "I don't know" most of the time and "I'm relatively sure" rarely.

7 The STRONG signal — consensus + time filter

The ★ **STRONG** signal is the system's single actionable output. It is the result of two stacked filters, both derived from backtests.

Filter 1 — multi-horizon consensus

A single horizon at HIGH tier is ~54%. But when **≥3 of the 4 horizons are simultaneously HIGH and agree on direction**, accuracy rises:

Horizons HIGH & agreeing	BankNifty	Minutes
≥ 1	53.8%	2445
≥ 2	55.9%	1031
≥ 3	59.0%	314
≥ 4 (all agree)	60.4%	67

The rule ships at `STRONG_MIN_ALIGN = 3`.

Honesty note (reconciled 30 Jul 2026). The table above is an *early held-out validation* cut, and the ≥4 row is only 67 minutes. On **live logged sessions** the STRONG signal comes in around **~54%** (and the model's own stored `strong_acc ≈ 54%`). Treat **~54%** as the honest, conservative STRONG number — the 59–60% here was a smaller, rosier sample. Also note: "N horizons agreeing on direction" (any tier) is a looser metric than "N horizons HIGH *and* agreeing", so it reads a few points lower again. Different cuts, all honest — but **~54% is the one to quote**.

Filter 2 — the midday dead-zone

Backtesting the 30-minute HIGH tier by time of day revealed a consistent pattern across both indices:

Window	BankNifty	Nifty50
Morning 09:15–11:00	55.1%	56.8%
Midday 11:00–13:30	47.9%	52.6%
Afternoon 13:30–15:30	54.7%	55.3%

Midday is the model's weakest window — BankNifty actually drops **below a coin toss**. Therefore STRONG is **suppressed between 11:00 and 13:30** (`DEAD_ZONE`). During those hours the UI shows a distinct "WAIT — midday dead-zone" banner even when the horizons agree.

```
strong = (align >= 3) AND (time not in 11:00–13:30)
signal = {strong, dir, align, acc, window}    # window: "prime" | "midday"
```

Net effect. Out of ~375 forecast minutes per session, STRONG fires only a handful of times — precisely when several models agree with high confidence in the reliable hours. Every other minute is an honest WAIT.

8 The ORB daily engine

The Opening-Range Breakout engine (`nifty_live.py`) is the project's original trading strategy, separate from the minute-by-minute forecaster. It was long described as the "money engine" — but an honest full-history backtest (§11) does **not** support that. It is documented here as designed; the honest performance verdict is in §11.

The rules

1. **Bias at the open.** The daily model sets a LONG/SHORT bias with a probability. If confidence < 58%, the whole day is **NO-TRADE** (unless the BankNifty adaptive unlock triggers).
2. **09:15–09:45 build the opening range** (its high and low). No trading.
3. **Entry:** a 1-minute close beyond the range in the bias direction.
4. **Stop:** the opposite side of the range. **Target:** 1.5R.
5. **Exits:** stop, target, or the 15:15 square-off — whichever first.
6. **Limits:** BankNifty up to 5 trades, entries until 14:30; Nifty50 up to 3, entries until 13:30. (Both tuned by backtest — extending Nifty50 losses.)

Adaptive unlock (BankNifty only)

On borderline-confidence days, a breakout is taken only if **4 of 4 intraday indicators** (RSI, EMA, MACD, VWAP) align with it; otherwise the day is skipped. (Only these four count — a July 2026 fix stopped the expanded 7-chip market-state card from loosening this gate to "4 of 7.")

Honest verdict (supersedes the old "money engine" claim). On the full 17-month / 351-session walk-forward, even with the adaptive unlock and all tuned settings, the ORB engine has **no robust edge**: BankNifty **−12.6R**, Nifty50 **+0.9R** (flat), and both LOSE on the training half while only profiting on the recent test half — regime luck, not an edge. Win-rate ~44–49% at 1.5:1 is break-even before costs and negative after. **Do not trade this with real money.** See §11.

9 The AI layer — Gemini analyst & co-pilot

`ai_analyst.py` adds two Gemini-powered panels. **They do not affect the forecast numbers** — the models run entirely without them. They are human-readable context only.

Panel	Endpoint	Output
AI Technical Analyst	/api/ai	Reads the live snapshot (day stats, opening range, prior levels, indicators, hit-rate) → structure, levels that matter, confirm/invalidate scenarios. Never a buy/sell signal.
AI Co-pilot	/api/copilot	Strict-JSON verdict: stance (LEANS-LONG / SHORT / STAND-ASIDE), conviction 1–5, read, wait-for. Conviction 4–5 only when horizons + indicators + hit-rates agree.

Model: `gemini-flash-latest` (free tier). Responses are cached ~10 minutes to stay inside the free daily quota. If the quota is exhausted, only these two panels degrade — the forecaster is unaffected.

10 The dashboard — routes, APIs, UI

`nifty_dashboard.py` runs a threaded HTTP server on `localhost:8181`. Per-index worker threads stream data, feed the engines, and update in-memory `STATE`; the browser polls JSON APIs.

Pages

Route	Page	Purpose
<code>/</code>	<code>dashboard.html</code>	The Beast — live candles, auto + manual paper trades, P&L tiles, trade log, indicator chips
<code>/forecasts</code>	<code>forecast_feed.html</code>	Forecast Window — 4-horizon cards, tiers, STRONG/WAIT banner, per-minute history with ✓/✗
<code>/quantum</code>	<code>quantum_live.html</code>	Market-state view — 7 indicator chips

JSON APIs

Endpoint	Returns
<code>/api/state</code>	Live candles, trades, P&L, indicators per index
<code>/api/tick</code>	Latest price (polled at 250 ms for smooth candles)
<code>/api/forecasts</code>	Current multi-horizon forecast, signal, today's hit-rate, backtest accuracy, history
<code>/api/ai</code> · <code>/api/copilot</code>	Gemini analyst text / co-pilot verdict

Modes: market open (Mon–Fri 09:15–15:30) → **LIVE**; otherwise **CLOSED** (last session shown statically). Forecasts are generated only in LIVE mode during market hours (a triple guard prevents stray after-hours rows).

11 Backtesting & honest results

Run `python3 backtest.py`. Methodology: train on all sessions before a cutoff, test on the last 45 held-out sessions, no lookahead, thresholds calibrated on train only. Below are the real out-of-sample hit-rates.

BankNifty (out-of-sample)

Horizon	All signals	HIGH tier	MEDIUM	LOW
5m	52.0%	54%	49%	52%
15m	51.1%	52%	51%	50%
30m	50.1%	56%	49%	47%
60m	48.6%	54%	45%	47%

Nifty50 (out-of-sample)

Horizon	All signals	HIGH tier	MEDIUM	LOW
5m	49.8%	51%	50%	48%
15m	50.0%	54%	47%	49%
30m	49.7%	56%	48%	47%
60m	51.8%	54%	53%	49%

Reading these numbers. "All signals" hovers around 50% — a coin toss, as expected. The **HIGH tier consistently beats it by 4–6 points**. Stacking the STRONG consensus + time filter on top lifts the actionable subset to ~54–59%. Those are honest hit-rates for the model's *direction* — but see the trading verdict below: a good hit-rate is not the same as making money.

ORB trading backtest — the honest verdict

The ORB strategy was run over the **full 17-month / 351-session** dataset (walk-forward daily bias), including the tuned per-index settings and BankNifty's adaptive unlock. Results, split into a training half (first 60%) and a test half (last 40%):

Index	Train (first 60%)	Test (last 40%)	Full 351
BankNifty	-15.2R (43% win)	+2.6R (53%)	-12.6R
Nifty50	-2.8R (46% win)	+3.7R (52%)	+0.9R (flat)

No robust edge. Both indices **lose on the training half** and only profit on the recent test half — the signature of regime luck, not a real edge (a genuine edge is positive on both). A parameter sweep found **no** setting positive on both halves. Win-rate ~44–49% at 1.5 : 1 is break-even before costs; realistic ~0.15R/trade transaction costs push it negative. The old "positive-R money engine" claim came from a short, lucky 60-day window. **Conclusion: do not trade this with real money — use it as an information and paper-trading tool.**

12 What was tested and removed

The graveyard is as important as the codebase. Each item below was a plausible idea that **failed an honest backtest** and was removed — documented so nobody re-adds it on a hunch.

Idea	Result	Verdict
Rapid-fire 5-min ML scalper (many trades/day)	Lost money in all 18 configurations	Removed
Extra features: VIX, GARCH, cross-index, cointegration	Train/serve skew (built from 2yr history at train, only intraday live); no reliable gain	Removed
Candlestick patterns (doji, engulfing, hammer) as model inputs	Helped neither index	UI-only
Chart-pattern-essence features (swings, range, squeeze, breakout)	BankNifty flat; Nifty50 mixed (+3.7 one horizon, -5.5 another = overfit noise)	Rejected
Meta-labeling + triple-barrier (López de Prado)	Rigorous multi-horizon, threshold on validation: base game deeply negative, meta couldn't find enough good trades	Rejected
ORB engine as a real-money strategy	Full 351-session walk-forward: no robust edge (loses on train, regime-luck on test); break-even before costs, negative after	Info-only
Time-of-day / regime filter on STRONG	Midday dead-zone real & consistent (BN 48%); good hours 55–57%	Shipped ✓
Multi-horizon HIGH consensus (≥ 3 agree)	Lifts BankNifty to ~59–60%	Shipped ✓

Ideas floated, not yet tested

- **Conviction-based position sizing** — bet more on STRONG, nothing on WAIT (improves profit more than accuracy).
- **Gap / news-day stand-aside filter** — the one live loss (−1.08R) was a gap-down news day.
- **Daily loss limit** — stop after N losing trades to protect capital.

13 APIs reference

Angel One SmartAPI (primary)

- Library: `smartapi-python` ; config: `angel_config.json` (`api_key`, `client_code`, `pin`, `totp_secret`).
- Auto-login: PIN + a 6-digit code generated locally from the TOTP secret via `pyotp` — **no SMS OTP**, so it works abroad.
- Candles: `getCandleData` (throttled ~2s). Ticks: `SmartWebSocketV2` , mode 1 LTP, `exchangeType` 1, price in paise/100.
- Index tokens: BankNifty `99926009` , Nifty50 `99926000` , India VIX `99926017` .
- Test: `python3 angel_data.py` .

Upstox (optional backup)

- Config: `upstox_config.json` ; OAuth token expires ~3:30 AM daily and needs SMS OTP.
- Commands: `python3 upstox_data.py login` → open URL → `token <code>` .
- **Historical option chain:** Upstox *Expired Instruments* API — requires Upstox Plus (currently free to activate, ~6 months history). Not used until it passes a backtest.
- `upstox_autologin.py` : a Playwright + `pyotp` stub to automate the daily login (WIP — for a friend's India-based account).

Gemini & Yahoo

- Gemini: `gemini_config.json` , model `gemini-flash-latest` , free tier, ~10-min cache. Powers only the AI panels.
- Yahoo: `yfinance` , no key. Fallback feed; avoid frequent polling (rate-limited).

Security. Config files hold live credentials — never print, commit, or share them. If a key is ever exposed in chat/logs, regenerate it in the provider's developer console.

14 Daily operating guide

1. **Any time (optional):** pre-train models — `python3 train_models.py` .
2. **At/after 09:15 on a weekday:** `python3 nifty_dashboard.py` . Angel logs in automatically; the browser opens to `localhost:8181` .
3. Open `/forecasts` . Watch the banner:
 -  **STRONG** BUY/SELL → the actionable moment (act on your own judgement + risk rules).
 - **WAIT** → do nothing. This is most of the day, by design.
 - **WAIT — midday dead-zone** → models agree but it is the weak 11:00–13:30 window.
4. Cross-check with the Beast (/): consensus, indicator chips, paper P&L, AI Co-pilot.
5. Respect position sizing (1% risk), the 15:15 square-off, and NO-TRADE days.
6. Any time, verify honesty: `python3 backtest.py` .

15 Risk management & limitations

Hard limits of this system

- **Accuracy ceiling ~59%.** No amount of tuning reaches 70–90%. Index direction is genuinely near-random at short horizons.
- **Warm-up.** Forecasts need ~1.5 hours of candles before the rolling indicators (20-bar windows) are valid.
- **Free-data quirks.** Yahoo lags; Angel revises the final bar to the official close (a live tick and the official close can differ slightly).
- **Regime dependence.** Trained on Feb 2025–Jul 2026. A market regime very unlike that period will degrade accuracy until retrained.

Survival rules

1. Risk 1% of capital per trade. **Never raise it.** A 4–6 loss streak is normal; at 10% risk that is account death.
2. Winners target 1.5R, losers cut at 1R — the math, not the hit-rate, is the edge.
3. Only act on STRONG, in the good hours. WAIT is a decision.
4. Paper-trade ≥ 20 sessions before real money. Only risk what you can lose entirely.
5. Nothing here is financial advice. Forecasting is probabilistic.

16 Statistical methods — what we kept & removed

Beyond the classic indicators, the project experimented with heavier statistical / econometric methods. True to the iron rule, each was **tested, not assumed**. Some survived and are live features; the more sophisticated ones did not and were removed. This section documents both.

16.1 Kept — statistical features that survived testing

These are among the 13 live features. They stayed because they are **computed identically in training and live** (no skew) and earned their place in the walk-forward backtest.

Method	What it measures	How it's computed
Hurst exponent	Whether the market is <i>trending</i> ($H > 0.5$), <i>mean-reverting</i> / <i>choppy</i> ($H < 0.5$), or a random walk ($H \approx 0.5$). The model behaves differently in each regime.	Rescaled-range (R/S) analysis over a rolling 20-bar window: $\log(R/S) / \log(20)$.
Parkinson volatility	Intraday volatility using the high–low range — more efficient than close-to-close because it uses the full bar.	$\sqrt{(\text{mean}(\ln(H/L)^2) / (4 \cdot \ln 2))}$ over 12 bars.
Regime volatility	How wild the market is right now — a simple statistical "temperature."	Rolling standard deviation of 1-bar returns (20 bars).

16.2 Removed — sophisticated statistics that failed the test

These sounded powerful and dominated the old engine's feature-importance list (it once claimed "statistical features are the top 6"). But an honest walk-forward exposed two fatal problems: a **train/serve mismatch** (they were built from 2 years of continuous history at training time, but only today's intraday bars at live time — so they meant different things) and **no reliable accuracy gain**.

Method	The idea	Why it was removed
GARCH(1,1)	An econometric model that forecasts volatility on the principle that volatility clusters — calm follows calm, turbulence follows turbulence.	Train/serve skew (rolling fit needs long history, absent intraday); slow; no measurable edge.
Cointegration spread z-score	Measures how far BankNifty and Nifty have drifted from their usual relationship — a pairs-trading / mean-reversion signal.	The normalisation referenced the series' start, which differed wildly between the 2-year train window and a single live day; did not help direction.
India VIX features	The market's "fear index" (raw level + rolling percentile) as a sentiment input.	Helped inconsistently across horizons; part of the skewed "extras" set. Note: VIX data is still cached, just not fed to the model.

The lesson. Sophistication is not edge. GARCH and cointegration are genuine, respected techniques — and they still failed here, because a method only earns a place if it survives an honest, train/serve-consistent backtest. The plain statistical features that stayed (Hurst, Parkinson, regime volatility) won precisely because they are simple and consistent, not because they are fancy.

17 File-by-file reference & glossary

Key modules

Module	Public surface
forecast_engine.py	ForecastEngine(name, capital, risk).forecast(candles, day) → dict with h (per-horizon), signal (STRONG), consensus; .backtest_acc, .tier_info, .strong_acc. Constants: HORIZONS, BASE_COLS, STRONG_MIN_ALIGN, DEAD_ZONE.
nifty_live.py	ORBEngine.on_minute(...), build_daily_features, walk_forward_bias, rsi, flat, TICKERS, FEATURE_SETS.
angel_data.py	AngelFeed(cfg).candles(name, interval), WebSocket tick stream, TOTP auto-login.
ai_analyst.py	briefing(snapshot), copilot(snapshot).

Glossary

Term	Meaning
Horizon	How far ahead a forecast looks: 5, 15, 30 or 60 minutes.
Tier	Confidence bucket (HIGH / MEDIUM / LOW) with a measured hit-rate.
STRONG	≥3 horizons HIGH and agreeing, in the good hours. The one actionable signal.
R	One unit of risk. +1.5R means a win of 1.5× the amount risked.
ORB	Opening-Range Breakout — the daily strategy.
Walk-forward	Backtest that only ever trains on the past and tests on the future — no lookahead.
Dead-zone	11:00–13:30, when the model is weakest and STRONG is suppressed.
TOTP	Time-based one-time password — a code generated locally from a secret, no SMS needed.